

Specyfikacja Implementacyjna

Narzędzie do pozycjonowania grafów (Java)

Zespół:

Polina Vyshnia,
Andrei Udavichenka.

Data: 21 maja 2026

DOKUMENT OPISUJE WEWNĘTRZNĄ BUDOWĘ PROGRAMU, ARCHITEKTURĘ PAKIETÓW ORAZ KLUCZOWE KLASY SYSTEMU WIZUALIZACJI I PODZIAŁU GRAFÓW ZAIMPLEMENTOWANYCH W JĘZYKU JAVA.

Spis treści

1	Wprowadzenie i technologie	2
2	Architektura systemu i podział na pakiety	2
3	Specyfikacja pakietów i klas	2
3.1	Pakiet GUI_Project	2
3.2	Pakiet GUI_Project.io	3
3.3	Pakiet GUI_Project.integration	3
3.4	Pakiet GUI_Project.view	3
3.5	Pakiet GUI_Project.presenter	3
4	Specyfikacja wyjątków i obsługa błędów	3
4.1	Błędy operacji wejścia/wyjścia (I/O) i parsowania	4
4.2	Błędy integracji z modulem C	4
4.3	Walidacja interfejsu i spójność stanu	4

1 Wprowadzenie i technologie

Niniejsza dokumentacja stanowi przewodnik po wewnętrznej budowie programu. Aplikacja została napisana w języku Java i służy do wczytywania, wizualizacji oraz podziału grafów.

Program jest w pełni zgodny z modulem obliczeniowym napisanym wcześniej w języku C. Odczytuje dane wygenerowane przez ten moduł bez żadnych konfliktów. Do stworzenia interfejsu graficznego (GUI) wykorzystano standardową bibliotekę Java Swing, a do rysowania grafów mechanizmy Graphics2D.

2 Architektura systemu i podział na pakiety

W celu zapewnienia wysokiej elastyczności kodu, niskiego poziomu powiązań (Low Coupling) oraz możliwości łatwego wprowadzania zmian wymagań, aplikację zaprojektowano zgodnie z paradygmatem MVP. Architekturę systemu podzielono na pięć dedykowanych pakietów:

- GUI_Project.model – struktury danych i konfiguracje procesów.
- GUI_Project.io – operacje dyskowe, parsowanie plików tekstowych i binarnych.
- GUI_Project.integration – komunikacja z zewnętrznymi procesami i uruchamianie modułu C.
- GUI_Project.view – komponenty graficzne Swing odpowiadające wyłącznie za prezentację.
- GUI_Project.presenter – logika sterująca interfejsem i pośrednicząca w wymianie danych.

3 Specyfikacja pakietów i klas

- Main – klasa główna zawierająca metodę uruchomieniową. Odpowiada za przekazanie sterowania do Event Dispatch Thread (EDT) i utworzenie obiektów warstwy prezentacji.

3.1 Pakiet GUI_Project

Reprezentuje stan aplikacji i struktury danych. Klasy w tym pakiecie nie zawierają logiki biznesowej.

- Node – model wierzchołka przechowujący jego identyfikator oraz współrzędne kartezjańskie X i Y.
- Edge – model krawędzi zawierający identyfikatory węzłów skrajnych, nazwę krawędzi oraz jej wagę.
- OutputGraph – kontener grupujący listy obiektów Node oraz Edge w jedną strukturę grafu.
- ProcessingConfig – klasa konfiguracyjna przechowująca ścieżki plików oraz wybrane parametry uruchomieniowe.

- `AlgorithmType` – typ wyliczeniowy (Enum) definiujący dostępne algorytmy (np. `Tutte`, `SpectralLayout`).
- `FileFormat` – typ wyliczeniowy definiujący formaty zapisu (`TEXT`, `BINARY`).

3.2 Pakiet `GUI_Project.io`

- `GraphFileReader` – odpowiada za odczyt danych. Implementuje metody przetwarzające pliki tekstowe oraz pliki binarne przy użyciu mechanizmu `ByteBuffer` w formacie `little-endian` w celu zachowania zgodności z modułem C.

3.3 Pakiet `GUI_Project.integration`

- `GraphProcessor` – interfejs definiujący kontrakt dla modułów przetwarzających grafy.
- `CExternalProcessor` – implementacja interfejsu `GraphProcessor`. Wykorzystuje klasę `ProcessBuilder` do niskopoziomowego uruchomienia skompilowanego pliku wykonywalnego C z odpowiednimi flagami systemowymi.

3.4 Pakiet `GUI_Project.view`

Warstwa odpowiedzialna za reprezentację graficzną. Komponenty są pasywne i nie podejmują decyzji o przepływie danych.

- `MainFrame` – główne okno aplikacji (`JFrame`) zawierające strukturę menu (`JMenuBar`) do wyboru plików.
- `GraphPanel` – autorski komponent (`JPanel`) realizujący rysowanie grafu poprzez nadpisaną metodę `paintComponent` z włączonym antyaliasingiem.
- `ToolbarPanel` – panel boczny zawierający elementy interaktywne do wprowadzania parametrów podziału.

3.5 Pakiet `GUI_Project.presenter`

- `MainPresenter` – centralny łącznik systemu. Przechwytuje zdarzenia z warstwy `View`, inicjuje odczyt plików przez pakiet `IO`, zleca obliczenia do pakietu `Integration`, a następnie przekazuje przetworzony obiekt `OutputGraph` z powrotem do `View` w celu aktualizacji obrazu.

4 Specyfikacja wyjątków i obsługa błędów

Aby zapobiec awaryjnemu zamykaniu się aplikacji, wszystkie krytyczne operacje (takie jak odczyt plików, konwersja danych czy uruchamianie zewnętrznych procesów) zostały zabezpieczone za pomocą bloków `try-catch`. Przechwycone wyjątki są obsługiwane w warstwie `Presentera`, co pozwala na pokazanie użytkownikowi czytelnego komunikatu i kontynuowanie pracy programu.

4.1 Błędy operacji wejścia/wyjścia (I/O) i parsowania

- **Błąd konwersji danych (NumberFormatException):** Występuje w bloku try-catch, gdy w pliku tekstowym lub panelu bocznym zamiast liczb znajdują się litery. Blok catch natychmiast przerywa operację i aplikacja wyświetla komunikat o błędnym formacie, nie psując stanu wczytanego grafu.
- **Nieoczekiwany koniec pliku (EOFException):** Zgłaszany przy czytaniu uszkodzonego pliku binarnego. Przechwycenie tego wyjątku zapobiega tworzeniu niepełnych obiektów Node w pamięci programu.
- **Błąd dostępu do pliku (IOException):** Przechwytywany podczas próby zapisu do folderu bez uprawnień, lub przy odczycie pliku, który został np. zablokowany przez system operacyjny.

4.2 Błędy integracji z modulem C

- **Błąd wykonania (Exit Code != 0):** Jeśli zewnętrzny moduł obliczeniowy (napisany w C) zwróci błąd w trakcie obliczeń, aplikacja Java przechwytuje ten kod wyjścia i informuje o niepowodzeniu, nie wyłączając głównego okna.
- **Brak pliku wykonywalnego:** Próba uruchomienia modułu poprzez ProcessBuilder, gdy plik C nie istnieje w danym folderze, rzuca wyjątek, który jest łapany w bloku catch i logowany jako błąd konfiguracji środowiska.

4.3 Walidacja interfejsu i spójność stanu

- **Niepoprawne parametry podziału:** Aplikacja zapobiega błędom matematycznym poprzez blokowanie podziału grafu, jeśli w panelu użytkownik wpisze wartości ujemne dla marginesu lub spróbuje podzielić graf na zero klastrów.
- **Ochrona przed NullPointerException:** Aplikacja dezaktywuje (wygasza) przycisk zapisu rozwiązania aż do momentu, gdy poprawny obiekt OutputGraph zostanie pomyślnie załadowany do pamięci.